In [1]:
```
!pip install requests
!pip install pandas
!pip install numpy
```

```
Requirement already satisfied: requests in c:\users\michael\anaconda3a\lib\site-
packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\michael\anac
onda3a\lib\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\michael\anaconda3a\lib\s
ite-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\michael\anaconda3a
\lib\site-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\michael\anaconda3a
\lib\site-packages (from requests) (2025.1.31)
Requirement already satisfied: pandas in c:\users\michael\anaconda3a\lib\site-pa
ckages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\michael\anaconda3a\lib
\site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\michael\anacon
da3a\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\michael\anaconda3a\lib\s
ite-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\michael\anaconda3a\lib
\site-packages (from pandas) (2023.3)
Requirement already satisfied: six>=1.5 in c:\users\michael\anaconda3a\lib\site-
packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Requirement already satisfied: numpy in c:\users\michael\anaconda3a\lib\site-pac
kages (1.26.4)
```

In [2]:
```
# Requests allows us to make HTTP requests which we will use to get data from ar
import requests
# Pandas is a software library written for the Python programming language for d
import pandas as pd
# NumPy is a library for the Python programming language, adding support for lar
import numpy as np
# Datetime is a library that allows us to represent dates
import datetime
```

In [3]:
```
# Setting this option will print all collumns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)
```

In [4]:
```
BoosterVersion = []

def getBoosterVersion(data):
    for rocket_id in data['rocket']:
        if rocket_id:
            try:
                response = requests.get(f"https://api.spacexdata.com/v4/rockets/
                response.raise_for_status()
                rocket_info = response.json()
                BoosterVersion.append(rocket_info.get('name', 'Unknown'))
            except Exception as e:
```

```
                print(f"Failed to fetch data for rocket ID {rocket_id}: {e}")
                BoosterVersion.append('Unknown')
```

In [5]:
```
Longitude = []
Latitude = []
LaunchSite = []

def getLaunchSite(data):
    for launchpad_id in data['launchpad']:
        if launchpad_id:
            try:
                response = requests.get(f"https://api.spacexdata.com/v4/launchpa
                response.raise_for_status()
                launch_info = response.json()
                Longitude.append(launch_info.get('longitude'))
                Latitude.append(launch_info.get('latitude'))
                LaunchSite.append(launch_info.get('name', 'Unknown'))
            except Exception as e:
                print(f"Error fetching launchpad {launchpad_id}: {e}")
                Longitude.append(None)
                Latitude.append(None)
                LaunchSite.append('Unknown')
```

In [6]:
```
PayloadMass = []
Orbit = []

def getPayloadData(data):
    for payload_id in data['payloads']:
        if payload_id:
            try:
                response = requests.get(f"https://api.spacexdata.com/v4/payloads
                response.raise_for_status()
                payload_info = response.json()
                PayloadMass.append(payload_info.get('mass_kg', None))
                Orbit.append(payload_info.get('orbit', 'Unknown'))
            except Exception as e:
                print(f"Error fetching payload {payload_id}: {e}")
                PayloadMass.append(None)
                Orbit.append('Unknown')
```

In [7]:
```
Block = []
ReusedCount = []
Serial = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []

def getCoreData(data):
    for core in data['cores']:
        if core and core.get('core'):
            try:
                response = requests.get(f"https://api.spacexdata.com/v4/cores/{
```

```
                response.raise_for_status()
                core_info = response.json()
                Block.append(core_info.get('block'))
                ReusedCount.append(core_info.get('reuse_count'))
                Serial.append(core_info.get('serial'))
            except Exception as e:
                print(f"Error fetching core {core['core']}: {e}")
                Block.append(None)
                ReusedCount.append(None)
                Serial.append(None)
        else:
            Block.append(None)
            ReusedCount.append(None)
            Serial.append(None)

        # These fields are directly in the dataset's 'cores' sub-dictionary
        Outcome.append(str(core.get('landing_success')) + ' ' + str(core.get('la
        Flights.append(core.get('flight'))
        GridFins.append(core.get('gridfins'))
        Reused.append(core.get('reused'))
        Legs.append(core.get('legs'))
        LandingPad.append(core.get('landpad'))
```

In [8]:
```
import requests
import pandas as pd

# Fetch the data from the SpaceX API
spacex_url = "https://api.spacexdata.com/v4/launches/past"
response = requests.get(spacex_url)

# Convert to JSON and then to a DataFrame
data_json = response.json()
data = pd.json_normalize(data_json)

# Quick peek at the dataset
print(data.shape)
data.head()
```

```
(187, 43)
```

Out[8]:

	static_fire_date_utc	static_fire_date_unix	net	window	ro
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	0.0	5e9d0d95eda69955f709
1	None	NaN	False	0.0	5e9d0d95eda69955f709
2	None	NaN	False	0.0	5e9d0d95eda69955f709
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	0.0	5e9d0d95eda69955f709

	static_fire_date_utc	static_fire_date_unix	net	window	ro
4	None	NaN	False	0.0	5e9d0d95eda69955f709c

In [9]:
```
response = requests.get(spacex_url)
```

In [10]:
```
if response.status_code == 200:
    print("Data fetched successfully!")
else:
    print(f"Failed to fetch data. Status code: {response.status_code}")
```

```
Data fetched successfully!
```

In [11]:
```
data_json = response.json()
```

In [14]:
```
data = pd.json_normalize(data_json)
```

In [12]:
```
import requests
import pandas as pd

# Step 1: Fetch the data from the SpaceX API
spacex_url = "https://api.spacexdata.com/v4/launches/past"
response = requests.get(spacex_url)

# Step 2: Check if the request was successful
if response.status_code == 200:
    print("✅ Data fetched successfully!")
    data_json = response.json()
else:
    print(f"❌ Failed to fetch data. Status code: {response.status_code}")

# Step 3: Convert JSON to DataFrame
data = pd.json_normalize(data_json)

# Step 4: Display basic info
print("\n📊 Data shape:", data.shape)
print("\n📋 Columns:")
print(data.columns.tolist())

# Step 5: Peek at the first few rows
```

```
data.head()
```

```
✅ Data fetched successfully!

📊 Data shape: (187, 43)

📋 Columns:
['static_fire_date_utc', 'static_fire_date_unix', 'net', 'window', 'rocket', 'su
ccess', 'failures', 'details', 'crew', 'ships', 'capsules', 'payloads', 'launchp
ad', 'flight_number', 'name', 'date_utc', 'date_unix', 'date_local', 'date_preci
sion', 'upcoming', 'cores', 'auto_update', 'tbd', 'launch_library_id', 'id', 'fa
irings.reused', 'fairings.recovery_attempt', 'fairings.recovered', 'fairings.shi
ps', 'links.patch.small', 'links.patch.large', 'links.reddit.campaign', 'links.r
eddit.launch', 'links.reddit.media', 'links.reddit.recovery', 'links.flickr.smal
l', 'links.flickr.original', 'links.presskit', 'links.webcast', 'links.youtube_i
d', 'links.article', 'links.wikipedia', 'fairings']
```

Out[12]:

	static_fire_date_utc	static_fire_date_unix	net	window	ro
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	0.0	5e9d0d95eda69955f709
1	None	NaN	False	0.0	5e9d0d95eda69955f709
2	None	NaN	False	0.0	5e9d0d95eda69955f709
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	0.0	5e9d0d95eda69955f709

	static_fire_date_utc	static_fire_date_unix	net	window	ro
4	None	NaN	False	0.0	5e9d0d95eda69955f709c

In [13]:
```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cl
```

In [14]:
```
import requests
import pandas as pd

# Step 1: Fetch the static JSON file
response = requests.get(static_json_url)

# Step 2: Check if the request was successful
if response.status_code == 200:
    print("✅ Data fetched successfully!")
    data_json = response.json()
else:
    print(f"❌ Failed to fetch data. Status code: {response.status_code}")

# Step 3: Convert JSON to DataFrame
data = pd.json_normalize(data_json)

# Step 4: Display basic info
print("\n📊 Data shape:", data.shape)
print("\n📋 Columns:")
print(data.columns.tolist())

# Step 5: Peek at the first few rows
data.head()
```

```
✅ Data fetched successfully!

📊 Data shape: (107, 42)

📋 Columns:
['static_fire_date_utc', 'static_fire_date_unix', 'tbd', 'net', 'window', 'rocke
t', 'success', 'details', 'crew', 'ships', 'capsules', 'payloads', 'launchpad',
'auto_update', 'failures', 'flight_number', 'name', 'date_utc', 'date_unix', 'da
te_local', 'date_precision', 'upcoming', 'cores', 'id', 'fairings.reused', 'fair
ings.recovery_attempt', 'fairings.recovered', 'fairings.ships', 'links.patch.sma
ll', 'links.patch.large', 'links.reddit.campaign', 'links.reddit.launch', 'links
.reddit.media', 'links.reddit.recovery', 'links.flickr.small', 'links.flickr.ori
ginal', 'links.presskit', 'links.webcast', 'links.youtube_id', 'links.article',
'links.wikipedia', 'fairings']
```

Out[14]:

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	False	0.0	5e9d0d95eda699
1	None	NaN	False	False	0.0	5e9d0d95eda699
2	None	NaN	False	False	0.0	5e9d0d95eda699
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	False	0.0	5e9d0d95eda699

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
4	None	NaN	False	False	0.0	5e9d0d95eda699

In [15]:
```
response=requests.get(static_json_url)
```

In [19]:
```
response.status_code
```

Out[19]: 200

In [16]:
```
if response.status_code == 200:
    print("✅ Data fetched successfully!")
    data_json = response.json()  # Convert the response to JSON
else:
    print(f"❌ Failed to fetch data. Status code: {response.status_code}")
```

```
✅ Data fetched successfully!
```

In [17]:
```
data = pd.json_normalize(data_json)
```

In [18]:
```
print("\n📊 Data shape:", data.shape)
print("\n📋 Columns:")
print(data.columns.tolist())

# Preview the first few rows
data.head()
```

```
📊 Data shape: (107, 42)

📋 Columns:
['static_fire_date_utc', 'static_fire_date_unix', 'tbd', 'net', 'window', 'rocke
t', 'success', 'details', 'crew', 'ships', 'capsules', 'payloads', 'launchpad',
'auto_update', 'failures', 'flight_number', 'name', 'date_utc', 'date_unix', 'da
te_local', 'date_precision', 'upcoming', 'cores', 'id', 'fairings.reused', 'fair
ings.recovery_attempt', 'fairings.recovered', 'fairings.ships', 'links.patch.sma
ll', 'links.patch.large', 'links.reddit.campaign', 'links.reddit.launch', 'links
.reddit.media', 'links.reddit.recovery', 'links.flickr.small', 'links.flickr.ori
ginal', 'links.presskit', 'links.webcast', 'links.youtube_id', 'links.article',
'links.wikipedia', 'fairings']
```

Out[18]:

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	False	0.0	5e9d0d95eda699
1	None	NaN	False	False	0.0	5e9d0d95eda699
2	None	NaN	False	False	0.0	5e9d0d95eda699
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	False	0.0	5e9d0d95eda699

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
4	None	NaN	False	False	0.0	5e9d0d95eda699

In [19]:
```
import requests
import pandas as pd

# URL of the static JSON file
static_json_url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain

# Step 1: Fetch the static JSON file from the URL
response = requests.get(static_json_url)

# Step 2: Check if the request was successful
if response.status_code == 200:
    print("✅ Data fetched successfully!")
    data_json = response.json()  # Convert the response to JSON
else:
    print(f"❌ Failed to fetch data. Status code: {response.status_code}")

# Step 3: Normalize the JSON and convert to a DataFrame
data = pd.json_normalize(data_json)

# Step 4: Display DataFrame info
print("\n📊 Data shape:", data.shape)  # Displays the number of rows and column
print("\n📋 Columns:")
print(data.columns.tolist())  # Lists all the columns

# Step 5: Display first few rows
data.head()  # Shows a preview of the first few rows
```

```
✅ Data fetched successfully!

📊 Data shape: (107, 42)

📋 Columns:
['static_fire_date_utc', 'static_fire_date_unix', 'tbd', 'net', 'window', 'rocke
t', 'success', 'details', 'crew', 'ships', 'capsules', 'payloads', 'launchpad',
'auto_update', 'failures', 'flight_number', 'name', 'date_utc', 'date_unix', 'da
te_local', 'date_precision', 'upcoming', 'cores', 'id', 'fairings.reused', 'fair
ings.recovery_attempt', 'fairings.recovered', 'fairings.ships', 'links.patch.sma
ll', 'links.patch.large', 'links.reddit.campaign', 'links.reddit.launch', 'links
.reddit.media', 'links.reddit.recovery', 'links.flickr.small', 'links.flickr.ori
ginal', 'links.presskit', 'links.webcast', 'links.youtube_id', 'links.article',
'links.wikipedia', 'fairings']
```

Out[19]:

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	False	0.0	5e9d0d95eda699
1	None	NaN	False	False	0.0	5e9d0d95eda699
2	None	NaN	False	False	0.0	5e9d0d95eda699
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	False	0.0	5e9d0d95eda699

	static_fire_date_utc	static_fire_date_unix	tbd	net	window	
4	None	NaN	False	False	0.0	5e9d0d95eda699

In [20]:
```
# Lets take a subset of our dataframe keeping only the features we want and the
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_

# We will remove rows with multiple cores because those are falcon rockets with
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single v
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracti
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

In [21]:
```
import requests
import pandas as pd
import datetime

# Global variables to store results
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
```

```
Latitude = []

# Function to get booster name (or version) from the rocket ID
def getBoosterVersion(rocket_id):
    response = requests.get(f"https://api.spacexdata.com/v4/rockets/{rocket_id}"
    if response.status_code == 200:
        return response.json()['name']
    return None

# Function to get payload mass and orbit from the payload ID
def getPayloadData(payload_id):
    response = requests.get(f"https://api.spacexdata.com/v4/payloads/{payload_id
    if response.status_code == 200:
        payload_data = response.json()
        return payload_data.get('mass_kg', None), payload_data.get('orbit', None
    return None, None

# Function to get launchpad info (name, longitude, latitude) from the launchpad
def getLaunchPadData(launchpad_id):
    response = requests.get(f"https://api.spacexdata.com/v4/launchpads/{launchpa
    if response.status_code == 200:
        launchpad_data = response.json()
        return launchpad_data['name'], launchpad_data['longitude'], launchpad_da
    return None, None, None

# Function to get core data (outcome, landing type, etc.) from the core ID
def getCoreData(core_id):
    response = requests.get(f"https://api.spacexdata.com/v4/cores/{core_id}")
    if response.status_code == 200:
        core_data = response.json()
        return {
            'outcome': str(core_data.get('landing_success', None)) + ' ' + str(
            'landing_type': core_data.get('landing_type', None),
            'flights': core_data.get('flight', None),
            'gridfins': core_data.get('gridfins', None),
            'reused': core_data.get('reused', None),
            'legs': core_data.get('legs', None),
            'landing_pad': core_data.get('landpad', None),
            'block': core_data.get('block', None),
            'reuse_count': core_data.get('reuse_count', None),
            'serial': core_data.get('serial', None)
        }
    return None

# Iterate over the dataframe and fill the global lists
for index, row in data.iterrows():
    # Extract rocket, payload, launchpad, and core IDs
    rocket_id = row['rocket']
    payload_id = row['payloads']
    launchpad_id = row['launchpad']
    core_id = row['cores']

    # Get booster version (name) from the rocket
    BoosterVersion.append(getBoosterVersion(rocket_id))

    # Get payload mass and orbit
```

```
    mass, orbit = getPayloadData(payload_id)
    PayloadMass.append(mass)
    Orbit.append(orbit)

    # Get launchpad name, longitude, and latitude
    site_name, lon, lat = getLaunchPadData(launchpad_id)
    LaunchSite.append(site_name)
    Longitude.append(lon)
    Latitude.append(lat)

    # Get core data (landing success, type, etc.)
    core_data = getCoreData(core_id)
    if core_data:
        Outcome.append(core_data['outcome'])
        LandingType.append(core_data['landing_type'])
        Flights.append(core_data['flights'])
        GridFins.append(core_data['gridfins'])
        Reused.append(core_data['reused'])
        Legs.append(core_data['legs'])
        LandingPad.append(core_data['landing_pad'])
        Block.append(core_data['block'])
        ReusedCount.append(core_data['reuse_count'])
        Serial.append(core_data['serial'])

# Create a new DataFrame with the extracted information
new_data = pd.DataFrame({
    'flight_number': data['flight_number'],
    'date_utc': data['date_utc'],
    'date': data['date'],
    'booster_version': BoosterVersion,
    'payload_mass': PayloadMass,
    'payload_orbit': Orbit,
    'launch_site': LaunchSite,
    'longitude': Longitude,
    'latitude': Latitude,
    'outcome': Outcome,
    'landing_type': LandingType,
    'flights': Flights,
    'gridfins': GridFins,
    'reused': Reused,
    'legs': Legs,
    'landing_pad': LandingPad,
    'block': Block,
    'reuse_count': ReusedCount,
    'serial': Serial
})

# Display the first few rows of the new DataFrame
print(new_data.head())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[21], line 112
     98         Serial.append(core_data['serial'])
    100 # Create a new DataFrame with the extracted information
    101 new_data = pd.DataFrame({
    102     'flight_number': data['flight_number'],
    103     'date_utc': data['date_utc'],
    104     'date': data['date'],
    105     'booster_version': BoosterVersion,
    106     'payload_mass': PayloadMass,
    107     'payload_orbit': Orbit,
    108     'launch_site': LaunchSite,
    109     'longitude': Longitude,
    110     'latitude': Latitude,
    111     'outcome': Outcome,
--> 112     'landing_type': LandingType,
    113     'flights': Flights,
    114     'gridfins': GridFins,
    115     'reused': Reused,
    116     'legs': Legs,
    117     'landing_pad': LandingPad,
    118     'block': Block,
    119     'reuse_count': ReusedCount,
    120     'serial': Serial
    121 })
    123 # Display the first few rows of the new DataFrame
    124 print(new_data.head())

NameError: name 'LandingType' is not defined
```

In [22]:

```python
# Global variables to store results
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []
LandingType = []  # Initialize LandingType as an empty list

# Function to get booster name (or version) from the rocket ID
def getBoosterVersion(rocket_id):
    response = requests.get(f"https://api.spacexdata.com/v4/rockets/{rocket_id}"
    if response.status_code == 200:
        return response.json()['name']
    return None
```

```
# Function to get payload mass and orbit from the payload ID
def getPayloadData(payload_id):
    response = requests.get(f"https://api.spacexdata.com/v4/payloads/{payload_id
    if response.status_code == 200:
        payload_data = response.json()
        return payload_data.get('mass_kg', None), payload_data.get('orbit', None
    return None, None

# Function to get launchpad info (name, longitude, latitude) from the launchpad
def getLaunchPadData(launchpad_id):
    response = requests.get(f"https://api.spacexdata.com/v4/launchpads/{launchpa
    if response.status_code == 200:
        launchpad_data = response.json()
        return launchpad_data['name'], launchpad_data['longitude'], launchpad_da
    return None, None, None

# Function to get core data (outcome, landing type, etc.) from the core ID
def getCoreData(core_id):
    response = requests.get(f"https://api.spacexdata.com/v4/cores/{core_id}")
    if response.status_code == 200:
        core_data = response.json()
        return {
            'outcome': str(core_data.get('landing_success', None)) + ' ' + str(c
            'landing_type': core_data.get('landing_type', None),
            'flights': core_data.get('flight', None),
            'gridfins': core_data.get('gridfins', None),
            'reused': core_data.get('reused', None),
            'legs': core_data.get('legs', None),
            'landing_pad': core_data.get('landpad', None),
            'block': core_data.get('block', None),
            'reuse_count': core_data.get('reuse_count', None),
            'serial': core_data.get('serial', None)
        }
    return None

# Iterate over the dataframe and fill the global lists
for index, row in data.iterrows():
    # Extract rocket, payload, launchpad, and core IDs
    rocket_id = row['rocket']
    payload_id = row['payloads']
    launchpad_id = row['launchpad']
    core_id = row['cores']

    # Get booster version (name) from the rocket
    BoosterVersion.append(getBoosterVersion(rocket_id))

    # Get payload mass and orbit
    mass, orbit = getPayloadData(payload_id)
    PayloadMass.append(mass)
    Orbit.append(orbit)

    # Get launchpad name, longitude, and latitude
    site_name, lon, lat = getLaunchPadData(launchpad_id)
    LaunchSite.append(site_name)
    Longitude.append(lon)
    Latitude.append(lat)
```

```
    # Get core data (landing success, type, etc.)
    core_data = getCoreData(core_id)
    if core_data:
        Outcome.append(core_data['outcome'])
        LandingType.append(core_data['landing_type'])  # Append landing_type to
        Flights.append(core_data['flights'])
        GridFins.append(core_data['gridfins'])
        Reused.append(core_data['reused'])
        Legs.append(core_data['legs'])
        LandingPad.append(core_data['landing_pad'])
        Block.append(core_data['block'])
        ReusedCount.append(core_data['reuse_count'])
        Serial.append(core_data['serial'])

# Create a new DataFrame with the extracted information
new_data = pd.DataFrame({
    'flight_number': data['flight_number'],
    'date
```

```
Cell In[22], line 100
  'date
  ^
SyntaxError: unterminated string literal (detected at line 100)
```

In [23]:

```
# Global variables to store results
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []
LandingType = []  # Initialize LandingType as an empty list

# Function to get booster name (or version) from the rocket ID
def getBoosterVersion(rocket_id):
    response = requests.get(f"https://api.spacexdata.com/v4/rockets/{rocket_id}"
    if response.status_code == 200:
        return response.json()['name']
    return None

# Function to get payload mass and orbit from the payload ID
def getPayloadData(payload_id):
    response = requests.get(f"https://api.spacexdata.com/v4/payloads/{payload_i
    if response.status_code == 200:
        payload_data = response.json()
        return payload_data.get('mass_kg', None), payload_data.get('orbit', Non
```

```
    return None, None

# Function to get launchpad info (name, longitude, latitude) from the launchpad
def getLaunchPadData(launchpad_id):
    response = requests.get(f"https://api.spacexdata.com/v4/launchpads/{launchpa
    if response.status_code == 200:
        launchpad_data = response.json()
        return launchpad_data['name'], launchpad_data['longitude'], launchpad_da
    return None, None, None

# Function to get core data (outcome, landing type, etc.) from the core ID
def getCoreData(core_id):
    response = requests.get(f"https://api.spacexdata.com/v4/cores/{core_id}")
    if response.status_code == 200:
        core_data = response.json()
        return {
            'outcome': str(core_data.get('landing_success', None)) + ' ' + str(c
            'landing_type': core_data.get('landing_type', None),
            'flights': core_data.get('flight', None),
            'gridfins': core_data.get('gridfins', None),
            'reused': core_data.get('reused', None),
            'legs': core_data.get('legs', None),
            'landing_pad': core_data.get('landpad', None),
            'block': core_data.get('block', None),
            'reuse_count': core_data.get('reuse_count', None),
            'serial': core_data.get('serial', None)
        }
    return None

# Iterate over the dataframe and fill the global lists
for index, row in data.iterrows():
    # Extract rocket, payload, launchpad, and core IDs
    rocket_id = row['rocket']
    payload_id = row['payloads']
    launchpad_id = row['launchpad']
    core_id = row['cores']

    # Get booster version (name) from the rocket
    BoosterVersion.append(getBoosterVersion(rocket_id))

    # Get payload mass and orbit
    mass, orbit = getPayloadData(payload_id)
    PayloadMass.append(mass)
    Orbit.append(orbit)

    # Get launchpad name, longitude, and latitude
    site_name, lon, lat = getLaunchPadData(launchpad_id)
    LaunchSite.append(site_name)
    Longitude.append(lon)
    Latitude.append(lat)

    # Get core data (landing success, type, etc.)
    core_data = getCoreData(core_id)
    if core_data:
        Outcome.append(core_data['outcome'])
        LandingType.append(core_data['landing_type'])  # Append landing_type to
```

```
        Flights.append(core_data['flights'])
        GridFins.append(core_data['gridfins'])
        Reused.append(core_data['reused'])
        Legs.append(core_data['legs'])
        LandingPad.append(core_data['landing_pad'])
        Block.append(core_data['block'])
        ReusedCount.append(core_data['reuse_count'])
        Serial.append(core_data['serial'])

# Create a new DataFrame with the extracted information
new_data = pd.DataFrame({
    'flight_number': data['flight_number'],
    'date_utc': data['date_utc'],
    'date': data['date'],
    'booster_version': BoosterVersion,
    'payload_mass': PayloadMass,
    'payload_orbit': Orbit,
    'launch_site': LaunchSite,
    'longitude': Longitude,
    'latitude': Latitude,
    'outcome': Outcome,
    'landing_type': LandingType,  # Now defined
    'flights': Flights,
    'gridfins': GridFins,
    'reused': Reused,
    'legs': Legs,
    'landing_pad': LandingPad,
    'block': Block,
    'reuse_count': ReusedCount,
    'serial': Serial
})

# Display the first few rows of the new DataFrame
print(new_data.head())
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[23], line 98
     95         Serial.append(core_data['serial'])
     97 # Create a new DataFrame with the extracted information
---> 98 new_data = pd.DataFrame({
     99     'flight_number': data['flight_number'],
    100     'date_utc': data['date_utc'],
    101     'date': data['date'],
    102     'booster_version': BoosterVersion,
    103     'payload_mass': PayloadMass,
    104     'payload_orbit': Orbit,
    105     'launch_site': LaunchSite,
    106     'longitude': Longitude,
    107     'latitude': Latitude,
    108     'outcome': Outcome,
    109     'landing_type': LandingType,  # Now defined
    110     'flights': Flights,
    111     'gridfins': GridFins,
    112     'reused': Reused,
    113     'legs': Legs,
    114     'landing_pad': LandingPad,
    115     'block': Block,
    116     'reuse_count': ReusedCount,
    117     'serial': Serial
    118 })
    120 # Display the first few rows of the new DataFrame
    121 print(new_data.head())

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:778, in DataFrame.__init
__(self, data, index, columns, dtype, copy)
    772     mgr = self._init_mgr(
    773         data, axes={"index": index, "columns": columns}, dtype=dtype, co
py=copy
    774     )
    776 elif isinstance(data, dict):
    777     # GH#38939 de facto copy defaults to False only in non-dict cases
--> 778     mgr = dict_to_mgr(data, index, columns, dtype=dtype, copy=copy, typ=
manager)
    779 elif isinstance(data, ma.MaskedArray):
    780     from numpy.ma import mrecords

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:503, in
dict_to_mgr(data, index, columns, dtype, typ, copy)
    499     else:
    500         # dtype check to exclude e.g. range objects, scalars
    501         arrays = [x.copy() if hasattr(x, "dtype") else x for x in array
s]
--> 503 return arrays_to_mgr(arrays, columns, index, dtype=dtype, typ=typ, conso
lidate=copy)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:114, in
arrays_to_mgr(arrays, columns, index, dtype, verify_integrity, typ, consolidate)
    111 if verify_integrity:
    112     # figure out the index, if necessary
    113     if index is None:
```

```
--> 114         index = _extract_index(arrays)
    115     else:
    116         index = ensure_index(index)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:677, in
_extract_index(data)
    675 lengths = list(set(raw_lengths))
    676 if len(lengths) > 1:
--> 677     raise ValueError("All arrays must be of the same length")
    679 if have_dicts:
    680     raise ValueError(
    681         "Mixing dicts with non-Series may lead to ambiguous ordering."
    682     )

ValueError: All arrays must be of the same length
```

In [24]:
```
print("Before applying getBoosterVersion:", BoosterVersion)
```

```
Before applying getBoosterVersion: ['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon
1', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Fal
con 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9',
'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon
9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Fal
con 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9',
'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon
9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Fal
con 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9',
'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon
9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Fal
con 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9',
'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon
9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Fal
con 9', 'Falcon 9', 'Falcon 9', 'Falcon 9']
```

In [25]:
```
BoosterVersion
```

Out[25]:
```
['Falcon 1',
 'Falcon 1',
 'Falcon 1',
 'Falcon 1',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
```

```
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9',
 'Falcon 9']
```

In [26]:

```
# Apply the getBoosterVersion function to fill the BoosterVersion list
getBoosterVersion(data)

# After the function is applied, the BoosterVersion list will be populated
print("Booster Version List:", BoosterVersion)
```

```
Booster Version List: ['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9
', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falc
on 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', '
Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9
', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falc
on 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', '
Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9
', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falc
on 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', '
Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9
', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falc
on 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', '
Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9
', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falcon 9', 'Falc
on 9', 'Falcon 9', 'Falcon 9']
```

In [27]:

```python
# Call getLaunchSite to get launchpad data
getLaunchSite(data)

# Call getPayloadData to get payload data (mass and orbit)
getPayloadData(data)

# Call getCoreData to get core data (landing, gridfins, etc.)
getCoreData(data)

# After these functions are called, the respective lists will be populated
print("Launch Site Data:", LaunchSite[0:5])
print("Payload Mass:", PayloadMass[0:5])
print("Payload Orbit:", Orbit[0:5])
print("Core Landing Outcome:", Outcome[0:5])
print("Core Flights:", Flights[0:5])
print("Core GridFins:", GridFins[0:5])
```

```
Launch Site Data: ['Kwajalein Atoll', 'Kwajalein Atoll', 'Kwajalein Atoll', 'Kwa
jalein Atoll', 'CCSFS SLC 40']
Payload Mass: [20, None, 165, 200, None]
Payload Orbit: ['LEO', 'LEO', 'LEO', 'LEO', 'LEO']
Core Landing Outcome: []
Core Flights: []
Core GridFins: []
```

In [28]:

```python
# Create the dictionary with the relevant data
launch_dict = {
    'FlightNumber': list(data['flight_number']),
    'Date': list(data['date']),
    'BoosterVersion': BoosterVersion,
    'PayloadMass': PayloadMass,
    'Orbit': Orbit,
    'LaunchSite': LaunchSite,
    'Outcome': Outcome,
    'Flights': Flights,
    'GridFins': GridFins,
    'Reused': Reused,
    'Legs': Legs,
    'LandingPad': LandingPad,
    'Block': Block,
    'ReusedCount': ReusedCount,
```

```
    'Serial': Serial,
    'Longitude': Longitude,
    'Latitude': Latitude
}

# Convert the dictionary into a DataFrame
launch_df = pd.DataFrame(launch_dict)

# Display the first few rows of the DataFrame
print(launch_df.head())
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[28], line 23
      2 launch_dict = {
      3     'FlightNumber': list(data['flight_number']),
      4     'Date': list(data['date']),
   (...)
     19     'Latitude': Latitude
     20 }
     22 # Convert the dictionary into a DataFrame
---> 23 launch_df = pd.DataFrame(launch_dict)
     25 # Display the first few rows of the DataFrame
     26 print(launch_df.head())

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:778, in DataFrame.__init
__(self, data, index, columns, dtype, copy)
    772     mgr = self._init_mgr(
    773         data, axes={"index": index, "columns": columns}, dtype=dtype, co
py=copy
    774     )
    776 elif isinstance(data, dict):
    777     # GH#38939 de facto copy defaults to False only in non-dict cases
--> 778     mgr = dict_to_mgr(data, index, columns, dtype=dtype, copy=copy, typ=
manager)
    779 elif isinstance(data, ma.MaskedArray):
    780     from numpy.ma import mrecords

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:503, in
dict_to_mgr(data, index, columns, dtype, typ, copy)
    499     else:
    500         # dtype check to exclude e.g. range objects, scalars
    501         arrays = [x.copy() if hasattr(x, "dtype") else x for x in array
s]
--> 503 return arrays_to_mgr(arrays, columns, index, dtype=dtype, typ=typ, conso
lidate=copy)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:114, in
arrays_to_mgr(arrays, columns, index, dtype, verify_integrity, typ, consolidate)
    111 if verify_integrity:
    112     # figure out the index, if necessary
    113     if index is None:
--> 114         index = _extract_index(arrays)
    115     else:
    116         index = ensure_index(index)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:677, in
_extract_index(data)
    675 lengths = list(set(raw_lengths))
    676 if len(lengths) > 1:
--> 677     raise ValueError("All arrays must be of the same length")
    679 if have_dicts:
    680     raise ValueError(
    681         "Mixing dicts with non-Series may lead to ambiguous ordering."
    682     )

ValueError: All arrays must be of the same length
```

In [29]:
```
print("Data length:", len(data))
print("BoosterVersion:", len(BoosterVersion))
print("PayloadMass:", len(PayloadMass))
print("Orbit:", len(Orbit))
print("LaunchSite:", len(LaunchSite))
print("Outcome:", len(Outcome))
print("Flights:", len(Flights))
print("GridFins:", len(GridFins))
print("Reused:", len(Reused))
print("Legs:", len(Legs))
print("LandingPad:", len(LandingPad))
print("Block:", len(Block))
print("ReusedCount:", len(ReusedCount))
print("Serial:", len(Serial))
print("Longitude:", len(Longitude))
print("Latitude:", len(Latitude))
```

```
Data length: 94
BoosterVersion: 94
PayloadMass: 94
Orbit: 94
LaunchSite: 188
Outcome: 0
Flights: 0
GridFins: 0
Reused: 0
Legs: 0
LandingPad: 0
Block: 0
ReusedCount: 0
Serial: 0
Longitude: 188
Latitude: 188
```

In [30]:
```
# Clear the global lists before repopulating
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []

# Re-run the data fetching functions
getBoosterVersion(data)
getLaunchSite(data)
getPayloadData(data)
getCoreData(data)
```

In [31]:
```
launch_dict = {
    'FlightNumber': list(data['flight_number']),
    'Date': list(data['date']),
    'BoosterVersion': BoosterVersion,
    'PayloadMass': PayloadMass,
    'Orbit': Orbit,
    'LaunchSite': LaunchSite,
    'Outcome': Outcome,
    'Flights': Flights,
    'GridFins': GridFins,
    'Reused': Reused,
    'Legs': Legs,
    'LandingPad': LandingPad,
    'Block': Block,
    'ReusedCount': ReusedCount,
    'Serial': Serial,
    'Longitude': Longitude,
    'Latitude': Latitude
}

launch_df = pd.DataFrame(launch_dict)
launch_df.head()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[31], line 21
      1 launch_dict = {
      2     'FlightNumber': list(data['flight_number']),
      3     'Date': list(data['date']),
   (...)
     18     'Latitude': Latitude
     19 }
---> 21 launch_df = pd.DataFrame(launch_dict)
     22 launch_df.head()

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:778, in DataFrame.__init
__(self, data, index, columns, dtype, copy)
    772     mgr = self._init_mgr(
    773         data, axes={"index": index, "columns": columns}, dtype=dtype, co
py=copy
    774     )
    776 elif isinstance(data, dict):
    777     # GH#38939 de facto copy defaults to False only in non-dict cases
--> 778     mgr = dict_to_mgr(data, index, columns, dtype=dtype, copy=copy, typ=
manager)
    779 elif isinstance(data, ma.MaskedArray):
    780     from numpy.ma import mrecords

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:503, in
dict_to_mgr(data, index, columns, dtype, typ, copy)
    499     else:
    500         # dtype check to exclude e.g. range objects, scalars
    501         arrays = [x.copy() if hasattr(x, "dtype") else x for x in array
s]
--> 503 return arrays_to_mgr(arrays, columns, index, dtype=dtype, typ=typ, conso
lidate=copy)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:114, in
arrays_to_mgr(arrays, columns, index, dtype, verify_integrity, typ, consolidate)
    111 if verify_integrity:
    112     # figure out the index, if necessary
    113     if index is None:
--> 114         index = _extract_index(arrays)
    115     else:
    116         index = ensure_index(index)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\construction.py:677, in
_extract_index(data)
    675 lengths = list(set(raw_lengths))
    676 if len(lengths) > 1:
--> 677     raise ValueError("All arrays must be of the same length")
    679 if have_dicts:
    680     raise ValueError(
    681         "Mixing dicts with non-Series may lead to ambiguous ordering."
    682     )

ValueError: All arrays must be of the same length
```

In [32]:
```
print("✅ Length of 'data':", len(data))
print("🚀 BoosterVersion:", len(BoosterVersion))
print("📦 PayloadMass:", len(PayloadMass))
print("🛰️ Orbit:", len(Orbit))
print("📍 LaunchSite:", len(LaunchSite))
print("🎯 Outcome:", len(Outcome))
print("🔁 Flights:", len(Flights))
print("🛠️ GridFins:", len(GridFins))
print("♻️ Reused:", len(Reused))
print("🦵 Legs:", len(Legs))
print("🛬 LandingPad:", len(LandingPad))
print("🧱 Block:", len(Block))
print("🔢 ReusedCount:", len(ReusedCount))
print("🔢 Serial:", len(Serial))
print("🌍 Longitude:", len(Longitude))
print("🌎 Latitude:", len(Latitude))
```

```
✅ Length of 'data': 94
🚀 BoosterVersion: 0
📦 PayloadMass: 0
🛰️ Orbit: 0
📍 LaunchSite: 94
🎯 Outcome: 0
🔁 Flights: 0
🛠️ GridFins: 0
♻️ Reused: 0
🦵 Legs: 0
🛬 LandingPad: 0
🧱 Block: 0
🔢 ReusedCount: 0
🔢 Serial: 0
🌍 Longitude: 94
🌎 Latitude: 94
```

In [33]:
```
import time

for rocket_id in data['rocket'][:5]:
    print(f"Fetching rocket: {rocket_id}")
    url = f"https://api.spacexdata.com/v4/rockets/{rocket_id}"
    response = requests.get(url)
    if response.status_code == 200:
        print("✅ Success:", response.json()['name'])
    else:
        print("❌ Failed to fetch:", response.status_code)
    time.sleep(1)  # sleep to avoid rate limits
```

```
Fetching rocket: 5e9d0d95eda69955f709d1eb
✅ Success: Falcon 1
Fetching rocket: 5e9d0d95eda69955f709d1eb
✅ Success: Falcon 1
Fetching rocket: 5e9d0d95eda69955f709d1eb
✅ Success: Falcon 1
Fetching rocket: 5e9d0d95eda69955f709d1eb
✅ Success: Falcon 1
Fetching rocket: 5e9d0d95eda69973a809d1ec
✅ Success: Falcon 9
```

In [34]:
```
def getBoosterVersion(data):
    for rocket_id in data['rocket']:
        try:
            response = requests.get(f"https://api.spacexdata.com/v4/rockets/{ro
            if response.status_code == 200:
                rocket_data = response.json()
                BoosterVersion.append(rocket_data.get('name', None))
            else:
                print(f"❌ API error for rocket ID {rocket_id}: {response.statu
                BoosterVersion.append(None)
        except Exception as e:
            print(f"🚨 Exception for rocket ID {rocket_id}: {e}")
            BoosterVersion.append(None)
```

In [35]:
```
BoosterVersion = []
```

In [36]:
```
getBoosterVersion(data)
```

In [37]:
```
print(len(BoosterVersion))  # Should be 94
print(BoosterVersion[:5])   # Sample output
```

```
94
['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9']
```

In [38]:
```
def getPayloadData(data):
    for payload_id in data['payloads']:
        try:
            response = requests.get(f"https://api.spacexdata.com/v4/payloads/{pa
            if response.status_code == 200:
                payload_data = response.json()
                PayloadMass.append(payload_data.get('mass_kg', None))
                Orbit.append(payload_data.get('orbit', None))
            else:
                print(f"❌ API error for payload ID {payload_id}: {response.sta
                PayloadMass.append(None)
                Orbit.append(None)
        except Exception as e:
            print(f"🚨 Exception for payload ID {payload_id}: {e}")
            PayloadMass.append(None)
            Orbit.append(None)
```

In [39]:
```
def getCoreData(data):
    for core in data['cores']:
        try:
            core_id = core.get('core')
            if core_id:
                response = requests.get(f"https://api.spacexdata.com/v4/cores/{
                if response.status_code == 200:
                    core_data = response.json()
                    Block.append(core_data.get('block', None))
                    ReusedCount.append(core_data.get('reuse_count', None))
                    Serial.append(core_data.get('serial', None))
                else:
                    Block.append(None)
                    ReusedCount.append(None)
```

```
                Serial.append(None)
            else:
                Block.append(None)
                ReusedCount.append(None)
                Serial.append(None)

            # Now handle the launch-specific info from the core dict
            Outcome.append(str(core.get('landing_success')) + " " + str(core.ge
            Flights.append(core.get('flight', None))
            GridFins.append(core.get('gridfins', None))
            Reused.append(core.get('reused', None))
            Legs.append(core.get('legs', None))
            LandingPad.append(core.get('landpad', None))
        except Exception as e:
            print(f"🚨 Exception in core data: {e}")
            Block.append(None)
            ReusedCount.append(None)
            Serial.append(None)
            Outcome.append(None)
            Flights.append(None)
            GridFins.append(None)
            Reused.append(None)
            Legs.append(None)
            LandingPad.append(None)
```

In [40]:
```
PayloadMass = []
Orbit = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []

getPayloadData(data)
getCoreData(data)
```

In [41]:
```
print(len(PayloadMass))
print(len(Outcome))
```

```
94
94
```

In [42]:
```
launch_dict = {
    'FlightNumber': list(data['flight_number']),
    'Date': list(data['date']),
    'BoosterVersion': BoosterVersion,
    'PayloadMass': PayloadMass,
    'Orbit': Orbit,
    'LaunchSite': LaunchSite,
    'Outcome': Outcome,
    'Flights': Flights,
    'GridFins': GridFins,
```

```
    'Reused': Reused,
    'Legs': Legs,
    'LandingPad': LandingPad,
    'Block': Block,
    'ReusedCount': ReusedCount,
    'Serial': Serial,
    'Longitude': Longitude,
    'Latitude': Latitude
}

# Convert to DataFrame
launch_df = pd.DataFrame(launch_dict)

# Show the first few rows
launch_df.head()
```

Out[42]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2006-03-24	Falcon 1	20.0	LEO	Kwajalein Atoll	Nor Nor
1	2	2007-03-21	Falcon 1	NaN	LEO	Kwajalein Atoll	Nor Nor
2	4	2008-09-28	Falcon 1	165.0	LEO	Kwajalein Atoll	Nor Nor
3	5	2009-07-13	Falcon 1	200.0	LEO	Kwajalein Atoll	Nor Nor
4	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	Nor Nor

In [43]:
```
launch_df.head()
```

Out[43]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2006-03-24	Falcon 1	20.0	LEO	Kwajalein Atoll	Nor Nor
1	2	2007-03-21	Falcon 1	NaN	LEO	Kwajalein Atoll	Nor Nor
2	4	2008-09-28	Falcon 1	165.0	LEO	Kwajalein Atoll	Nor Nor
3	5	2009-07-13	Falcon 1	200.0	LEO	Kwajalein Atoll	Nor Nor
4	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	Nor Nor

In [44]:
```
data_falcon9 = launch_df[launch_df['BoosterVersion'] != 'Falcon 1'].reset_index(
```

In [45]:
```
print("Shape of Falcon 9 data:", data_falcon9.shape)
```

```
print("Unique booster versions:", data_falcon9['BoosterVersion'].unique())
data_falcon9.head()
```

```
Shape of Falcon 9 data: (90, 17)
Unique booster versions: ['Falcon 9']
```

Out[45]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	Nor Nor
1	8	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	Nor Nor
2	10	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	Nor Nor
3	11	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	Fal Ocea
4	12	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	Nor Nor

In [46]:
```
data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
```

In [47]:
```
data_falcon9.head()
```

Out[47]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	Nor Nor
1	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	Nor Nor
2	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	Nor Nor
3	4	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	Fal Ocea
4	5	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	Nor Nor

In [48]:
```
print(data_falcon9)
```

```
    FlightNumber        Date BoosterVersion  PayloadMass Orbit    LaunchSite  \
0              1  2010-06-04       Falcon 9          NaN   LEO  CCSFS SLC 40
1              2  2012-05-22       Falcon 9        525.0   LEO  CCSFS SLC 40
2              3  2013-03-01       Falcon 9        677.0   ISS  CCSFS SLC 40
3              4  2013-09-29       Falcon 9        500.0    PO   VAFB SLC 4E
4              5  2013-12-03       Falcon 9       3170.0   GTO  CCSFS SLC 40
..           ...         ...            ...          ...   ...           ...
85            86  2020-09-03       Falcon 9      15600.0  VLEO    KSC LC 39A
86            87  2020-10-06       Falcon 9      15600.0  VLEO    KSC LC 39A
87            88  2020-10-18       Falcon 9      15600.0  VLEO    KSC LC 39A
88            89  2020-10-24       Falcon 9      15600.0  VLEO  CCSFS SLC 40
89            90  2020-11-05       Falcon 9       3681.0   MEO  CCSFS SLC 40

        Outcome  Flights  GridFins  Reused   Legs                LandingPad  \
0     None None        1     False   False  False                      None
1     None None        1     False   False  False                      None
2     None None        1     False   False  False                      None
3   False Ocean        1     False   False  False                      None
4     None None        1     False   False  False                      None
..          ...      ...       ...     ...    ...                       ...
85    True ASDS        2      True    True   True  5e9e3032383ecb6bb234e7ca
86    True ASDS        3      True    True   True  5e9e3032383ecb6bb234e7ca
87    True ASDS        6      True    True   True  5e9e3032383ecb6bb234e7ca
88    True ASDS        3      True    True   True  5e9e3033383ecbb9e534e7cc
89    True ASDS        1      True   False   True  5e9e3032383ecb6bb234e7ca

    Block  ReusedCount Serial   Longitude   Latitude
0     1.0            0  B0003  -80.577366  28.561857
1     1.0            0  B0005  -80.577366  28.561857
2     1.0            0  B0007  -80.577366  28.561857
3     1.0            0  B1003 -120.610829  34.632093
4     1.0            0  B1004  -80.577366  28.561857
..    ...          ...    ...         ...        ...
85    5.0           12  B1060  -80.603956  28.608058
86    5.0           13  B1058  -80.603956  28.608058
87    5.0           12  B1051  -80.603956  28.608058
88    5.0           12  B1060  -80.577366  28.561857
89    5.0            8  B1062  -80.577366  28.561857

[90 rows x 17 columns]
```

In [49]:
```
data_falcon9.isnull().sum()
```

Out[49]:
```
FlightNumber       0
Date               0
BoosterVersion     0
PayloadMass        5
Orbit              0
LaunchSite         0
Outcome            0
Flights            0
GridFins           0
Reused             0
Legs               0
LandingPad        26
Block              0
ReusedCount        0
Serial             0
Longitude          0
Latitude           0
dtype: int64
```

In [50]:
```
mean_payload_mass = data_falcon9['PayloadMass'].mean()
print(f"Mean PayloadMass: {mean_payload_mass}")
```

```
Mean PayloadMass: 6123.547647058824
```

In [51]:
```
data_falcon9['PayloadMass'].replace(np.nan, mean_payload_mass, inplace=True)
```

```
C:\Users\Michael\AppData\Local\Temp\ipykernel_14188\2413704982.py:1: FutureWarni
ng: A value is trying to be set on a copy of a DataFrame or Series through chain
ed assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work beca
use the intermediate object on which we are setting values always behaves as a c
opy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met
hod({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to
perform the operation inplace on the original object.


  data_falcon9['PayloadMass'].replace(np.nan, mean_payload_mass, inplace=True)
```

In [52]:
```
data_falcon9['PayloadMass'].isnull().sum()
```

Out[52]:
```
0
```

In [53]:
```
data_falcon9.isnull().sum()
```

Out[53]:
```
FlightNumber       0
Date               0
BoosterVersion     0
PayloadMass        0
Orbit              0
LaunchSite         0
Outcome            0
Flights            0
GridFins           0
Reused             0
Legs               0
LandingPad        26
Block              0
ReusedCount        0
Serial             0
Longitude          0
Latitude           0
dtype: int64
```

In [54]:
```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

In [55]:
```
!pip3 install beautifulsoup4
!pip3 install requests
```

```
Requirement already satisfied: beautifulsoup4 in c:\users\michael\anaconda3a\lib
\site-packages (4.12.3)
Requirement already satisfied: soupsieve>1.2 in c:\users\michael\anaconda3a\lib
\site-packages (from beautifulsoup4) (2.5)
Requirement already satisfied: requests in c:\users\michael\anaconda3a\lib\site-
packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\michael\anac
onda3a\lib\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\michael\anaconda3a\lib\s
ite-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\michael\anaconda3a
\lib\site-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\michael\anaconda3a
\lib\site-packages (from requests) (2025.1.31)
```

In [56]:
```
import sys

import requests
from bs4 import BeautifulSoup
import re
import unicodedata
import pandas as pd
```

In [57]:
```
def date_time(table_cells):
    """
    This function returns the data and time from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
```

```
    This function returns the booster version from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=''.join([booster_version for i,booster_version in enumerate( table_cells
    return out

def landing_status(table_cells):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=[i for i in table_cells.strings][0]
    return out


def get_mass(table_cells):
    mass=unicodedata.normalize("NFKD", table_cells.text).strip()
    if mass:
        mass.find("kg")
        new_mass=mass[0:mass.find("kg")+2]
    else:
        new_mass=0
    return new_mass


def extract_column_from_header(row):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    if (row.br):
        row.br.extract()
    if row.a:
        row.a.extract()
    if row.sup:
        row.sup.extract()

    colunm_name = ' '.join(row.contents)

    # Filter the digit and empty names
    if not(colunm_name.strip().isdigit()):
        colunm_name = colunm_name.strip()
        return colunm_name
```

In [58]:
```
def get_mass(table_cells):
    mass = unicodedata.normalize("NFKD", table_cells.text).strip()
    if "kg" in mass:
        new_mass = mass[:mass.find("kg") + 2]
    else:
        new_mass = "0 kg"
    return new_mass
```

In [59]:
```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Fa
```

In [60]:
```
response = requests.get(static_url)
```

```
soup = BeautifulSoup(response.text, "html.parser")

# Find all launch tables (they are typically class="wikitable")
tables = soup.find_all("table", {"class": "wikitable"})

# Example: process just the first table for now
launch_data = []

for row in tables[0].find_all("tr")[1:]:  # skip header
    cells = row.find_all("td")
    if len(cells) > 0:
        launch_date, launch_time = date_time(cells[0])
        booster = booster_version(cells[1])
        status = landing_status(cells[-1])
        payload_mass = get_mass(cells[5])  # Example index - adjust as needed

        launch_data.append({
            "Date": launch_date,
            "Time": launch_time,
            "Booster": booster,
            "Mass": payload_mass,
            "Landing Status": status
        })

# Convert to DataFrame
df = pd.DataFrame(launch_data)
print(df.head())
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[60], line 14
     12 if len(cells) > 0:
     13     launch_date, launch_time = date_time(cells[0])
---> 14     booster = booster_version(cells[1])
     15     status = landing_status(cells[-1])
     16     payload_mass = get_mass(cells[5])  # Example index - adjust as neede
d

IndexError: list index out of range
```

In [61]:
```
for row in tables[0].find_all("tr")[1:]:
    cells = row.find_all("td")

    # Ensure there are enough cells to unpack safely (adjust based on your needs
    if len(cells) >= 6:
        try:
            launch_date, launch_time = date_time(cells[0])
            booster = booster_version(cells[1])
            payload_mass = get_mass(cells[5])
            status = landing_status(cells[-1])

            launch_data.append({
                "Date": launch_date,
                "Time": launch_time,
                "Booster": booster,
                "Mass": payload_mass,
```

```
                "Landing Status": status
            })
        except Exception as e:
            print(f"Skipping row due to error: {e}")
```

In [62]:
```
import requests

# The static URL of the Falcon 9 and Falcon Heavy launches Wikipedia page
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Fa

# Perform GET request
response = requests.get(static_url)

# Check the status
print("Status Code:", response.status_code)
```

```
Status Code: 200
```

In [63]:
```
from bs4 import BeautifulSoup

# Create a BeautifulSoup object from the response content
soup = BeautifulSoup(response.text, "html.parser")

# Optional: Check the first 500 characters of the parsed HTML to verify it's wor
print(soup.prettify()[:500])  # Prints the first 500 characters in a nicely form
```

```
<!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled vector-featur
e-language-in-main-page-header-disabled vector-feature-page-tools-pinned-disable
d vector-feature-toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disable
d vector-feature-limited-width-clientpref-1 vector-feature-limited-width-conten
t-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearanc
e-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-da
y vect
```

In [64]:
```
# Print the title of the page
print("Page Title:", soup.title.string)
```

```
Page Title: List of Falcon 9 and Falcon Heavy launches - Wikipedia
```

In [65]:
```
# Find all tables on the Wikipedia page (class="wikitable" is typically used for
tables = soup.find_all("table", {"class": "wikitable"})

# Check how many tables we found
print(f"Number of tables found: {len(tables)}")

# Example: Let's extract the column names (headers) from the first table
headers = []
for th in tables[0].find_all("th"):
    # Extract text and clean up extra whitespace
    header_text = th.get_text(strip=True)
    headers.append(header_text)

# Print the column names (headers)
print("Column Names:", headers)
```

```
Number of tables found: 13
Column Names: ['Flight No.', 'Date andtime (UTC)', 'Version,Booster[b]', 'Launch
site', 'Payload[c]', 'Payload mass', 'Orbit', 'Customer', 'Launchoutcome', 'Boos
terlanding', '1', '2', '3', '4', '5', '6', '7']
```

In [66]:
```
# Use find_all to find all tables on the page
html_tables = soup.find_all("table")

# Check how many tables were found
print(f"Number of tables found: {len(html_tables)}")
```

```
Number of tables found: 25
```

In [67]:
```
# Access the third table (index 2) in the list of tables
first_launch_table = html_tables[2]

# Print the content of the third table
print(first_launch_table)
```

```
<table class="wikitable plainrowheaders collapsible" style="width: 100%;">
<tbody><tr>
<th scope="col">Flight No.
</th>
<th scope="col">Date and<br/>time (<a href="/wiki/Coordinated_Universal_Time" ti
tle="Coordinated Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="Lis
t of Falcon 9 first-stage boosters">Version,<br/>Booster</a> <sup class="referen
ce" id="cite_ref-booster_11-0"><a href="#cite_note-booster-11"><span class="cit
e-bracket">[</span>b<span class="cite-bracket">]</span></a></sup>
</th>
<th scope="col">Launch site
</th>
<th scope="col">Payload<sup class="reference" id="cite_ref-Dragon_12-0"><a href
="#cite_note-Dragon-12"><span class="cite-bracket">[</span>c<span class="cite-br
acket">]</span></a></sup>
</th>
<th scope="col">Payload mass
</th>
<th scope="col">Orbit
</th>
<th scope="col">Customer
</th>
<th scope="col">Launch<br/>outcome
</th>
<th scope="col"><a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon
9 first-stage landing tests">Booster<br/>landing</a>
</th></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">1
</th>
<td>4 June 2010,<br/>18:45
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-0"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B0003.1<sup class="reference" id="cite_ref-block_numbers_14-0"><a href="#cite
_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-br
acket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/Dragon_Spacecraft_Qualification_Unit" title="Dragon Spacecraf
t Qualification Unit">Dragon Spacecraft Qualification Unit</a>
</td>
<td>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/SpaceX" title="SpaceX">SpaceX</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success
```

```
</td>
<td class="table-failure" style="background: #FFC7C7; color:black; vertical-alig
n: middle; text-align: center;">Failure<sup class="reference" id="cite_ref-ns201
10930_15-0"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</spa
n>9<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_
ref-16"><a href="#cite_note-16"><span class="cite-bracket">[</span>10<span class
="cite-bracket">]</span></a></sup><br/><small>(parachute)</small>
</td></tr>
<tr>
<td colspan="9">First flight of Falcon 9 v1.0.<sup class="reference" id="cite_re
f-sfn20100604_17-0"><a href="#cite_note-sfn20100604-17"><span class="cite-bracke
t">[</span>11<span class="cite-bracket">]</span></a></sup> Used a boilerplate ve
rsion of Dragon capsule which was not designed to separate from the second stag
e.<small>(<a href="#First_flight_of_Falcon_9">more details below</a>)</small> At
tempted to recover the first stage by parachuting it into the ocean, but it burn
ed up on reentry, before the parachutes even deployed.<sup class="reference" id
="cite_ref-parachute_18-0"><a href="#cite_note-parachute-18"><span class="cite-b
racket">[</span>12<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">2
</th>
<td>8 December 2010,<br/>15:43<sup class="reference" id="cite_ref-spaceflightnow
_Clark_Launch_Report_19-0"><a href="#cite_note-spaceflightnow_Clark_Launch_Repor
t-19"><span class="cite-bracket">[</span>13<span class="cite-bracket">]</span></
a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-1"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B0004.1<sup class="reference" id="cite_ref-block_numbers_14-1"><a href="#cite
_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-br
acket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_Dragon" title="SpaceX Dragon">Dragon</a> <a class="mw-
redirect" href="/wiki/COTS_Demo_Flight_1" title="COTS Demo Flight 1">demo flight
C1</a><br/>(Dragon C101)
</td>
<td>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/w
iki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><style data-mw-deduplicate="TemplateStyles:r1126788409">.mw-parser-output .p
lainlist ol,.mw-parser-output .plainlist ul{line-height:inherit;list-style:non
e;margin:0;padding:0}.mw-parser-output .plainlist ol li,.mw-parser-output .plain
list ul li{margin-bottom:0}</style><div class="plainlist">
<ul><li><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Or
bital_Transportation_Services" title="Commercial Orbital Transportation Service
s">COTS</a>)</li>
<li><a href="/wiki/National_Reconnaissance_Office" title="National Reconnaissanc
e Office">NRO</a></li></ul>
```

```
</div>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success<sup class="reference" id="cite_ref-ns201
10930_15-1"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</spa
n>9<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-failure" style="background: #FFC7C7; color:black; vertical-alig
n: middle; text-align: center;">Failure<sup class="reference" id="cite_ref-ns201
10930_15-2"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</spa
n>9<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_
ref-20"><a href="#cite_note-20"><span class="cite-bracket">[</span>14<span class
="cite-bracket">]</span></a></sup><br/><small>(parachute)</small>
</td></tr>
<tr>
<td colspan="9">Maiden flight of <a class="mw-redirect" href="/wiki/Dragon_capsu
le" title="Dragon capsule">Dragon capsule</a>, consisting of over 3 hours of tes
ting thruster maneuvering and reentry.<sup class="reference" id="cite_ref-spacef
lightnow_Clark_unleashing_Dragon_21-0"><a href="#cite_note-spaceflightnow_Clark_
unleashing_Dragon-21"><span class="cite-bracket">[</span>15<span class="cite-bra
cket">]</span></a></sup> Attempted to recover the first stage by parachuting it
into the ocean, but it disintegrated upon reentry, before the parachutes were de
ployed.<sup class="reference" id="cite_ref-parachute_18-1"><a href="#cite_note-p
arachute-18"><span class="cite-bracket">[</span>12<span class="cite-bracket">]</
span></a></sup> <small>(<a href="#COTS_demo_missions">more details below</a>)</s
mall> It also included two <a href="/wiki/CubeSat" title="CubeSat">CubeSats</a
>,<sup class="reference" id="cite_ref-NRO_Taps_Boeing_for_Next_Batch_of_CubeSats
_22-0"><a href="#cite_note-NRO_Taps_Boeing_for_Next_Batch_of_CubeSats-22"><span
class="cite-bracket">[</span>16<span class="cite-bracket">]</span></a></sup> and
a wheel of <a href="/wiki/Brou%C3%A8re" title="Brouère">Brouère</a> cheese.
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">3
</th>
<td>22 May 2012,<br/>07:44<sup class="reference" id="cite_ref-BBC_new_era_23-0">
<a href="#cite_note-BBC_new_era-23"><span class="cite-bracket">[</span>17<span c
lass="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-2"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B0005.1<sup class="reference" id="cite_ref-block_numbers_14-2"><a href="#cite
_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-br
acket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_Dragon" title="SpaceX Dragon">Dragon</a> <a class="mw-
redirect" href="/wiki/Dragon_C2%2B" title="Dragon C2+">demo flight C2+</a><sup c
lass="reference" id="cite_ref-C2_24-0"><a href="#cite_note-C2-24"><span class="c
ite-bracket">[</span>18<span class="cite-bracket">]</span></a></sup><br/>(Dragon
C102)
</td>
<td>525 kg (1,157 lb)<sup class="reference" id="cite_ref-25"><a href="#cite_not
```

```
e-25"><span class="cite-bracket">[</span>19<span class="cite-bracket">]</span></
a></sup>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/w
iki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Orbita
l_Transportation_Services" title="Commercial Orbital Transportation Services">CO
TS</a>)
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success<sup class="reference" id="cite_ref-26"><
a href="#cite_note-26"><span class="cite-bracket">[</span>20<span class="cite-br
acket">]</span></a></sup>
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-alig
n: middle; white-space: nowrap; text-align: center;">No attempt
</td></tr>
<tr>
<td colspan="9">Dragon spacecraft demonstrated a series of tests before it was a
llowed to approach the <a href="/wiki/International_Space_Station" title="Intern
ational Space Station">International Space Station</a>. Two days later, it becam
e the first commercial spacecraft to board the ISS.<sup class="reference" id="ci
te_ref-BBC_new_era_23-1"><a href="#cite_note-BBC_new_era-23"><span class="cite-b
racket">[</span>17<span class="cite-bracket">]</span></a></sup> <small>(<a href=
"#COTS_demo_missions">more details below</a>)</small>
</td></tr>
<tr>
<th rowspan="3" scope="row" style="text-align:center;">4
</th>
<td rowspan="2">8 October 2012,<br/>00:35<sup class="reference" id="cite_ref-SFN
_LLog_27-0"><a href="#cite_note-SFN_LLog-27"><span class="cite-bracket">[</span>
21<span class="cite-bracket">]</span></a></sup>
</td>
<td rowspan="2"><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><
sup class="reference" id="cite_ref-MuskMay2012_13-3"><a href="#cite_note-MuskMay
2012-13"><span class="cite-bracket">[</span>7<span class="cite-bracket">]</span>
</a></sup><br/>B0006.1<sup class="reference" id="cite_ref-block_numbers_14-3"><a
href="#cite_note-block_numbers-14"><span class="cite-bracket">[</span>8<span cla
ss="cite-bracket">]</span></a></sup>
</td>
<td rowspan="2"><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape C
anaveral Space Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space
_Launch_Complex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_CRS-1" title="SpaceX CRS-1">SpaceX CRS-1</a><sup class
="reference" id="cite_ref-sxManifest20120925_28-0"><a href="#cite_note-sxManifes
t20120925-28"><span class="cite-bracket">[</span>22<span class="cite-bracke
t">]</span></a></sup><br/>(Dragon C103)
</td>
<td>4,700 kg (10,400 lb)
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/w
iki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Resupp
```

```
ly_Services" title="Commercial Resupply Services">CRS</a>)
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success
</td>
<td rowspan="2" style="background:#ececec; text-align:center;"><span class="nowr
ap">No attempt</span>
</td></tr>
<tr>
<td><a href="/wiki/Orbcomm_(satellite)" title="Orbcomm (satellite)">Orbcomm-OG2
</a><sup class="reference" id="cite_ref-Orbcomm_29-0"><a href="#cite_note-Orbcom
m-29"><span class="cite-bracket">[</span>23<span class="cite-bracket">]</span></
a></sup>
</td>
<td>172 kg (379 lb)<sup class="reference" id="cite_ref-gunter-og2_30-0"><a href
="#cite_note-gunter-og2-30"><span class="cite-bracket">[</span>24<span class="ci
te-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/Orbcomm" title="Orbcomm">Orbcomm</a>
</td>
<td class="table-partial" style="background: #FFB; color:black; vertical-align:
middle; text-align: center;">Partial failure<sup class="reference" id="cite_ref-
nyt-20121030_31-0"><a href="#cite_note-nyt-20121030-31"><span class="cite-bracke
t">[</span>25<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<td colspan="9">CRS-1 was successful, but the <a href="/wiki/Secondary_payload"
title="Secondary payload">secondary payload</a> was inserted into an abnormally
low orbit and subsequently lost. This was due to one of the nine <a href="/wiki/
SpaceX_Merlin" title="SpaceX Merlin">Merlin engines</a> shutting down during the
launch, and NASA declining a second reignition, as per <a href="/wiki/Internatio
nal_Space_Station" title="International Space Station">ISS</a> visiting vehicle
safety rules, the primary payload owner is contractually allowed to decline a se
cond reignition. NASA stated that this was because SpaceX could not guarantee a
high enough likelihood of the second stage completing the second burn successful
ly which was required to avoid any risk of secondary payload's collision with th
e ISS.<sup class="reference" id="cite_ref-OrbcommTotalLoss_32-0"><a href="#cite_
note-OrbcommTotalLoss-32"><span class="cite-bracket">[</span>26<span class="cit
e-bracket">]</span></a></sup><sup class="reference" id="cite_ref-sn20121011_3
3-0"><a href="#cite_note-sn20121011-33"><span class="cite-bracket">[</span>27<sp
an class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-3
4"><a href="#cite_note-34"><span class="cite-bracket">[</span>28<span class="cit
e-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">5
</th>
<td>1 March 2013,<br/>15:10
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-4"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B0007.1<sup class="reference" id="cite_ref-block_numbers_14-4"><a href="#cite
_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-br
```

```
acket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_CRS-2" title="SpaceX CRS-2">SpaceX CRS-2</a><sup class
="reference" id="cite_ref-sxManifest20120925_28-1"><a href="#cite_note-sxManifes
t20120925-28"><span class="cite-bracket">[</span>22<span class="cite-bracke
t">]</span></a></sup><br/>(Dragon C104)
</td>
<td>4,877 kg (10,752 lb)
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a class="m
w-redirect" href="/wiki/ISS" title="ISS">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Resupp
ly_Services" title="Commercial Resupply Services">CRS</a>)
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-alig
n: middle; white-space: nowrap; text-align: center;">No attempt
</td></tr>
<tr>
<td colspan="9">Last launch of the original Falcon 9 v1.0 <a href="/wiki/Launch_
vehicle" title="Launch vehicle">launch vehicle</a>, first use of the unpressuriz
ed trunk section of Dragon.<sup class="reference" id="cite_ref-sxf9_20110321_3
5-0"><a href="#cite_note-sxf9_20110321-35"><span class="cite-bracket">[</span>29
<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">6
</th>
<td>29 September 2013,<br/>16:00<sup class="reference" id="cite_ref-pa20130930_3
6-0"><a href="#cite_note-pa20130930-36"><span class="cite-bracket">[</span>30<sp
an class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.1" title="Falcon 9 v1.1">F9 v1.1</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-5"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B1003<sup class="reference" id="cite_ref-block_numbers_14-5"><a href="#cite_n
ote-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-brac
ket">]</span></a></sup>
</td>
<td><a class="mw-redirect" href="/wiki/Vandenberg_Air_Force_Base" title="Vandenb
erg Air Force Base">VAFB</a>,<br/><a href="/wiki/Vandenberg_Space_Launch_Complex
_4" title="Vandenberg Space Launch Complex 4">SLC-4E</a>
</td>
<td><a href="/wiki/CASSIOPE" title="CASSIOPE">CASSIOPE</a><sup class="reference"
id="cite_ref-sxManifest20120925_28-2"><a href="#cite_note-sxManifest20120925-2
8"><span class="cite-bracket">[</span>22<span class="cite-bracket">]</span></a>
</sup><sup class="reference" id="cite_ref-CASSIOPE_MDA_37-0"><a href="#cite_not
e-CASSIOPE_MDA-37"><span class="cite-bracket">[</span>31<span class="cite-bracke
t">]</span></a></sup>
```

```
</td>
<td>500 kg (1,100 lb)
</td>
<td><a href="/wiki/Polar_orbit" title="Polar orbit">Polar orbit</a> <a href="/wi
ki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/Maxar_Technologies" title="Maxar Technologies">MDA</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success<sup class="reference" id="cite_ref-pa201
30930_36-1"><a href="#cite_note-pa20130930-36"><span class="cite-bracket">[</spa
n>30<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-no2" style="background: #FFE3E3; color: black; vertical-align:
middle; text-align: center;">Uncontrolled<br/><small>(ocean)</small><sup class="
reference" id="cite_ref-ocean_landing_38-0"><a href="#cite_note-ocean_landing-3
8"><span class="cite-bracket">[</span>d<span class="cite-bracket">]</span></a></
sup>
</td></tr>
<tr>
<td colspan="9">First commercial mission with a private customer, first launch f
rom Vandenberg, and demonstration flight of Falcon 9 v1.1 with an improved 13-to
nne to LEO capacity.<sup class="reference" id="cite_ref-sxf9_20110321_35-1"><a h
ref="#cite_note-sxf9_20110321-35"><span class="cite-bracket">[</span>29<span cla
ss="cite-bracket">]</span></a></sup> After separation from the second stage carr
ying Canadian commercial and scientific satellites, the first stage booster perf
ormed a controlled reentry,<sup class="reference" id="cite_ref-39"><a href="#cit
e_note-39"><span class="cite-bracket">[</span>32<span class="cite-bracket">]</sp
an></a></sup> and an <a href="/wiki/Falcon_9_first-stage_landing_tests" title="F
alcon 9 first-stage landing tests">ocean touchdown test</a> for the first time.
This provided good test data, even though the booster started rolling as it near
ed the ocean, leading to the shutdown of the central engine as the roll depleted
it of fuel, resulting in a hard impact with the ocean.<sup class="reference" id
="cite_ref-pa20130930_36-2"><a href="#cite_note-pa20130930-36"><span class="cit
e-bracket">[</span>30<span class="cite-bracket">]</span></a></sup> This was the
first known attempt of a rocket engine being lit to perform a supersonic retro p
ropulsion, and allowed SpaceX to enter a public-private partnership with <a href
="/wiki/NASA" title="NASA">NASA</a> and its Mars entry, descent, and landing tec
hnologies research projects.<sup class="reference" id="cite_ref-40"><a href="#ci
te_note-40"><span class="cite-bracket">[</span>33<span class="cite-bracket">]</s
pan></a></sup> <small>(<a href="#Maiden_flight_of_v1.1">more details below</a
>)</small>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">7
</th>
<td>3 December 2013,<br/>22:41<sup class="reference" id="cite_ref-sfn_wwls201306
24_41-0"><a href="#cite_note-sfn_wwls20130624-41"><span class="cite-bracket">[</
span>34<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.1" title="Falcon 9 v1.1">F9 v1.1</a><br/>B1004
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
```

```
<td><a href="/wiki/SES-8" title="SES-8">SES-8</a><sup class="reference" id="cite
_ref-sxManifest20120925_28-3"><a href="#cite_note-sxManifest20120925-28"><span c
lass="cite-bracket">[</span>22<span class="cite-bracket">]</span></a></sup><sup
class="reference" id="cite_ref-spx-pr_42-0"><a href="#cite_note-spx-pr-42"><span
class="cite-bracket">[</span>35<span class="cite-bracket">]</span></a></sup><sup
class="reference" id="cite_ref-aw20110323_43-0"><a href="#cite_note-aw20110323-4
3"><span class="cite-bracket">[</span>36<span class="cite-bracket">]</span></a>
</sup>
</td>
<td>3,170 kg (6,990 lb)
</td>
<td><a href="/wiki/Geostationary_transfer_orbit" title="Geostationary transfer o
rbit">GTO</a>
</td>
<td><a class="mw-redirect" href="/wiki/SES_S.A." title="SES S.A.">SES</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success<sup class="reference" id="cite_ref-SNMis
sionStatus7_44-0"><a href="#cite_note-SNMissionStatus7-44"><span class="cite-bra
cket">[</span>37<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-alig
n: middle; white-space: nowrap; text-align: center;">No attempt<br/><sup class="
reference" id="cite_ref-sf10120131203_45-0"><a href="#cite_note-sf10120131203-4
5"><span class="cite-bracket">[</span>38<span class="cite-bracket">]</span></a>
</sup>
</td></tr>
<tr>
<td colspan="9">First <a href="/wiki/Geostationary_transfer_orbit" title="Geosta
tionary transfer orbit">Geostationary transfer orbit</a> (GTO) launch for Falcon
9,<sup class="reference" id="cite_ref-spx-pr_42-1"><a href="#cite_note-spx-pr-4
2"><span class="cite-bracket">[</span>35<span class="cite-bracket">]</span></a>
</sup> and first successful reignition of the second stage.<sup class="reference
" id="cite_ref-46"><a href="#cite_note-46"><span class="cite-bracket">[</span>39
<span class="cite-bracket">]</span></a></sup> SES-8 was inserted into a <a href
="/wiki/Geostationary_transfer_orbit" title="Geostationary transfer orbit">Supe
r-Synchronous Transfer Orbit</a> of 79,341 km (49,300 mi) in apogee with an <a h
ref="/wiki/Orbital_inclination" title="Orbital inclination">inclination</a> of 2
0.55° to the <a href="/wiki/Equator" title="Equator">equator</a>.
</td></tr></tbody></table>
```

In [68]:

```python
# Initialize an empty list to hold the column names
column_names = []

# Iterate through all <th> elements in the first_launch_table
for th in first_launch_table.find_all("th"):
    # Apply the provided extract_column_from_header() function to extract the co
    column_name = extract_column_from_header(th)

    # Append the non-empty column name to the list
    if column_name and len(column_name) > 0:
        column_names.append(column_name)

# Print the column names
print("Column Names:", column_names)
```

```
Column Names: ['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Pay
load mass', 'Orbit', 'Customer', 'Launch outcome']
```

In [69]:
```
print(column_names)
```

```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'O
rbit', 'Customer', 'Launch outcome']
```

In [70]:
```
# Initialize an empty dictionary with column names as keys
launch_dict = dict.fromkeys(column_names)

# Remove the irrelevant column
del launch_dict['Date and time ( )']

# Initialize each column with an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []

# Add the new columns
launch_dict['Version Booster'] = []
launch_dict['Booster landing'] = []
launch_dict['Date'] = []
launch_dict['Time'] = []

# Print the dictionary to check its structure
print(launch_dict)
```

```
{'Flight No.': [], 'Launch site': [], 'Payload': [], 'Payload mass': [], 'Orbit'
: [], 'Customer': [], 'Launch outcome': [], 'Version Booster': [], 'Booster land
ing': [], 'Date': [], 'Time': []}
```

In [71]:
```
extracted_row = 0

# Extract each table
for table_number, table in enumerate(soup.find_all('table', "wikitable plainrowl
    # Get table rows
    for rows in table.find_all("tr"):
        # Check if the first table heading corresponds to a launch number
        if rows.th:
            if rows.th.string:
                flight_number = rows.th.string.strip()
                flag = flight_number.isdigit()  # Check if the flight number is
        else:
            flag = False

        # Get table elements (tds)
        row = rows.find_all('td')

        # If it's a valid row (flag is True), process it
        if flag:
            extracted_row += 1
```

```
            # Flight Number
            launch_dict['Flight No.'].append(flight_number)

            # Date and Time
            datatimelist = date_time(row[0])
            # Date
            date = datatimelist[0].strip(',')
            launch_dict['Date'].append(date)
            # Time
            time = datatimelist[1]
            launch_dict['Time'].append(time)

            # Booster Version
            bv = booster_version(row[1])
            if not bv:
                bv = row[1].a.string  # In case booster version is nested inside
            launch_dict['Version Booster'].append(bv)

            # Launch Site
            launch_site = row[2].a.string if row[2].a else None  # Handle missi
            launch_dict['Launch site'].append(launch_site)

            # Payload
            payload = row[3].a.string if row[3].a else None  # Handle missing va
            launch_dict['Payload'].append(payload)

            # Payload Mass
            payload_mass = get_mass(row[4])  # Extracted using the provided func
            launch_dict['Payload mass'].append(payload_mass)

            # Orbit
            orbit = row[5].a.string if row[5].a else None  # Handle missing valu
            launch_dict['Orbit'].append(orbit)

            # Customer
            customer = row[6].a.string if row[6].a else None  # Handle missing v
            launch_dict['Customer'].append(customer)

            # Launch Outcome
            launch_outcome = list(row[7].strings)[0]  # Extract the first string
            launch_dict['Launch outcome'].append(launch_outcome)

            # Booster Landing
            booster_landing = landing_status(row[8])  # Extracted using the prov
            launch_dict['Booster landing'].append(booster_landing)

# At this point, launch_dict is populated with launch data
# Convert to DataFrame
import pandas as pd
launch_df = pd.DataFrame(launch_dict)

# Display the first few rows of the DataFrame
print(launch_df.head())
```

```
  Flight No. Launch site                                Payload Payload mass  \
0          1       CCAFS  Dragon Spacecraft Qualification Unit         0 kg
1          2       CCAFS                                Dragon         0 kg
2          3       CCAFS                                Dragon       525 kg
3          4       CCAFS                           SpaceX CRS-1     4,700 kg
4          5       CCAFS                           SpaceX CRS-2     4,877 kg

  Orbit Customer Launch outcome   Version Booster Booster landing  \
0   LEO   SpaceX      Success\n  F9 v1.07B0003.18         Failure
1   LEO     NASA        Success  F9 v1.07B0004.18         Failure
2   LEO     NASA        Success  F9 v1.07B0005.18    No attempt\n
3   LEO     NASA      Success\n  F9 v1.07B0006.18      No attempt
4   LEO     NASA      Success\n  F9 v1.07B0007.18    No attempt\n

              Date   Time
0      4 June 2010  18:45
1  8 December 2010  15:43
2      22 May 2012  07:44
3   8 October 2012  00:35
4     1 March 2013  15:10
```

In [72]:
```
# Create the DataFrame from the dictionary
df = pd.DataFrame({key: pd.Series(value) for key, value in launch_dict.items()})

# Display the first few rows of the DataFrame to verify
print(df.head())
```

```
  Flight No. Launch site                                Payload Payload mass  \
0          1       CCAFS  Dragon Spacecraft Qualification Unit         0 kg
1          2       CCAFS                                Dragon         0 kg
2          3       CCAFS                                Dragon       525 kg
3          4       CCAFS                           SpaceX CRS-1     4,700 kg
4          5       CCAFS                           SpaceX CRS-2     4,877 kg

  Orbit Customer Launch outcome   Version Booster Booster landing  \
0   LEO   SpaceX      Success\n  F9 v1.07B0003.18         Failure
1   LEO     NASA        Success  F9 v1.07B0004.18         Failure
2   LEO     NASA        Success  F9 v1.07B0005.18    No attempt\n
3   LEO     NASA      Success\n  F9 v1.07B0006.18      No attempt
4   LEO     NASA      Success\n  F9 v1.07B0007.18    No attempt\n

              Date   Time
0      4 June 2010  18:45
1  8 December 2010  15:43
2      22 May 2012  07:44
3   8 October 2012  00:35
4     1 March 2013  15:10
```

In [73]:
```
# Export the DataFrame to a CSV file
df.to_csv('spacex_web_scraped.csv', index=False)

# Confirm the CSV has been saved
print("CSV file 'spacex_web_scraped.csv' has been successfully saved.")
```

```
CSV file 'spacex_web_scraped.csv' has been successfully saved.
```

In [74]:
```
# Filter the dataframe to only include Falcon 9 launches
```

```
falcon_9_df = df[df['Payload'].str.contains('Falcon 9', na=False)]

# Display the filtered dataframe
print(falcon_9_df.head())
```

```
Empty DataFrame
Columns: [Flight No., Launch site, Payload, Payload mass, Orbit, Customer, Launc
h outcome, Version Booster, Booster landing, Date, Time]
Index: []
```

In [75]:
```
# Replace NaN values in 'Payload mass' with the mean of that column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[75], line 2
      1 # Replace NaN values in 'Payload mass' with the mean of that column
----> 2 df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)
      4 # Display the updated DataFrame to verify
      5 print(df['Payload mass'].head())

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:6549, in Series.mean(se
lf, axis, skipna, numeric_only, **kwargs)
   6541 @doc(make_doc("mean", ndim=1))
   6542 def mean(
   6543     self,
   (...)
   6547     **kwargs,
   6548 ):
-> 6549     return NDFrame.mean(self, axis, skipna, numeric_only, **kwargs)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:12420, in NDFrame.mean
(self, axis, skipna, numeric_only, **kwargs)
  12413 def mean(
  12414     self,
  12415     axis: Axis | None = 0,
   (...)
  12418     **kwargs,
  12419 ) -> Series | float:
> 12420     return self._stat_function(
  12421         "mean", nanops.nanmean, axis, skipna, numeric_only, **kwargs
  12422     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:12377, in NDFrame._sta
t_function(self, name, func, axis, skipna, numeric_only, **kwargs)
  12373 nv.validate_func(name, (), kwargs)
  12375 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 12377 return self._reduce(
  12378     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_only
  12379 )

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:6457, in Series._reduce
(self, op, name, axis, skipna, numeric_only, filter_type, **kwds)
   6452     # GH#47500 - change to TypeError to match other methods
   6453     raise TypeError(
   6454         f"Series.{name} does not allow {kwd_name}={numeric_only} "
   6455         "with non-numeric dtypes."
   6456     )
-> 6457 return op(delegate, skipna=skipna, **kwds)

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:147, in bottleneck_swit
ch.__call__.<locals>.f(values, axis, skipna, **kwds)
    145         result = alt(values, axis=axis, skipna=skipna, **kwds)
    146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kwds)
    149 return result

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:404, in _datetimelike_c
ompat.<locals>.new_func(values, axis, skipna, mask, **kwargs)
```

```
    401 if datetimelike and mask is None:
    402     mask = isna(values)
--> 404 result = func(values, axis=axis, skipna=skipna, mask=mask, **kwargs)
    406 if datetimelike:
    407     result = _wrap_results(result, orig_values.dtype, fill_value=iNaT)

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:720, in nanmean(values,
axis, skipna, mask)
    718 count = _get_counts(values.shape, mask, axis, dtype=dtype_count)
    719 the_sum = values.sum(axis, dtype=dtype_sum)
--> 720 the_sum = _ensure_numeric(the_sum)
    722 if axis is not None and getattr(the_sum, "ndim", False):
    723     count = cast(np.ndarray, count)

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:1701, in _ensure_numeri
c(x)
   1698 elif not (is_float(x) or is_integer(x) or is_complex(x)):
   1699     if isinstance(x, str):
   1700         # GH#44008, GH#36703 avoid casting e.g. strings to numeric
-> 1701         raise TypeError(f"Could not convert string '{x}' to numeric")
   1702     try:
   1703         x = float(x)

TypeError: Could not convert string '0 kg0 kg525 kg4,700 kg4,877 kg500 kg3,170 k
g3,325 kg2,296 kg1,316 kg4,535 kg4,428 kg2,216 kg2,395 kg570 kg4,159 kg1,898 kg4
,707 kg1,952 kg2,034 kg553 kg5,271 kg3,136 kg4,696 kg3,100 kg3,600 kg2,257 kg4,6
00 kg9,600 kg2,490 kg5,600 kg5,300 kg0 kg6,070 kg2,708 kg3,669 kg9,600 kg6,761 k
g3,310 kg475 kg4,990 kg9,600 kg5,200 kg3,500 kg2,205 kg9,600 kg0 kg4,230 kg2,150
kg6,092 kg9,600 kg2,647 kg362 kg3,600 kg6,460 kg5,384 kg2,697 kg7,075 kg9,600 kg
5,800 kg7,060 kg3,000 kg5,300 kg~4,000 kg2,500 kg4,400 kg9,600 kg4,850 kg12,055
kg2,495 kg13,620 kg4,200 kg2,268 kg6,500 kg15,600 kg2,617 kg6,956 kg15,600 kg12,
050 kg15,600 kg15,600 kg1,977 kg15,600 kg15,600 kg12,530 kg15,600 kg15,410 kg4,3
11 kg5,000–6,000 kg14,932 kg~15,440 kg3,130 kg15,600 kg15,600 kg15,600 kg15,600
kg4,311 kg~12,500 kg1,192 kg15,600 kg2,972 kg7,000 kg0 kg3,500 kg15,600 kg~5,000
kg15,600 kg15,600 kg15,600 kg15,600 kg15,600 kg15,600 kg15,600 kg~13,000 kg15,60
0 kg15,600 kg15,600 kg~14,000 kg15,600 kg3,328 kg7,000 kg' to numeric
```

In [76]:
```
# Remove 'kg' and commas, then convert to numeric values
df['Payload mass'] = df['Payload mass'].str.replace(' kg', '', regex=False)  # R
df['Payload mass'] = df['Payload mass'].str.replace(',', '', regex=False)  # Re

# Convert the column to numeric values (this will convert invalid parsing to NaN
df['Payload mass'] = pd.to_numeric(df['Payload mass'], errors='coerce')

# Replace NaN values with the mean of the column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

```
0       0.0
1       0.0
2     525.0
3    4700.0
4    4877.0
Name: Payload mass, dtype: float64
```

```
C:\Users\Michael\AppData\Local\Temp\ipykernel_14188\4205711417.py:9: FutureWarni
ng: A value is trying to be set on a copy of a DataFrame or Series through chain
ed assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work beca
use the intermediate object on which we are setting values always behaves as a c
opy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met
hod({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to
perform the operation inplace on the original object.


  df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)
```

In [77]:
```
# Export the DataFrame to a CSV file
df.to_csv('spacex_web_scraped.csv', index=False)

# Confirm the CSV has been saved
print("CSV file 'spacex_web_scraped.csv' has been successfully saved.")
```

```
CSV file 'spacex_web_scraped.csv' has been successfully saved.
```

In [78]:
```
import requests
import pandas as pd

# Step 1: Send GET request to the SpaceX API
url = "https://api.spacexdata.com/v4/launches"  # Example SpaceX API URL for la
response = requests.get(url)

# Step 2: Convert the response JSON into a DataFrame
launch_data = response.json()
df = pd.json_normalize(launch_data)

# Step 3: Extract the year from the 'static_fire_date_utc' column of the first r
# Convert 'static_fire_date_utc' to datetime format
df['static_fire_date_utc'] = pd.to_datetime(df['static_fire_date_utc'])

# Extract the year from the first row
first_row_year = df['static_fire_date_utc'].iloc[0].year

# Print the year
print("Year in the first row 'static_fire_date_utc':", first_row_year)
```

```
Year in the first row 'static_fire_date_utc': 2006
```

In [79]:
```
import requests
import pandas as pd

# Step 1: Send GET request to the SpaceX API
url = "https://api.spacexdata.com/v4/launches"  # SpaceX API URL for launches
response = requests.get(url)

# Step 2: Convert the response JSON into a DataFrame
launch_data = response.json()
df = pd.json_normalize(launch_data)

# Step 3: Filter for Falcon 9 launches (assuming the vehicle info is in a key co
```

```
# This assumes the column with the rocket type name is 'rocket' and Falcon 9 lau
falcon_9_df = df[df['rocket'].str.contains('Falcon 9', case=False, na=False)]

# Step 4: Remove Falcon 1 launches by filtering them out (assuming 'Falcon 1' is
falcon_9_df = falcon_9_df[~falcon_9_df['rocket'].str.contains('Falcon 1', case=F

# Step 5: Count the number of Falcon 9 launches
falcon_9_launch_count = len(falcon_9_df)

# Print the count of Falcon 9 launches after removing Falcon 1 launches
print(f"Number of Falcon 9 launches after removing Falcon 1 launches: {falcon_9_
```

```
Number of Falcon 9 launches after removing Falcon 1 launches: 0
```

In [80]:

```
import requests
import pandas as pd

# Step 1: Send GET request to the SpaceX API
url = "https://api.spacexdata.com/v4/launches"  # SpaceX API URL for launches
response = requests.get(url)

# Step 2: Convert the response JSON into a DataFrame
launch_data = response.json()
df = pd.json_normalize(launch_data)

# Step 3: Check for missing values in the 'landingPad' column
missing_values = df['landing_pad'].isna().sum()

# Print the number of missing values in the 'landingPad' column
print(f"Number of missing values in the 'landingPad' column: {missing_values}")
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.ge
t_loc(self, key)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\\_libs\\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

File pandas\\_libs\\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

KeyError: 'landing_pad'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[80], line 13
     10 df = pd.json_normalize(launch_data)
     12 # Step 3: Check for missing values in the 'landingPad' column
---> 13 missing_values = df['landing_pad'].isna().sum()
     15 # Print the number of missing values in the 'landingPad' column
     16 print(f"Number of missing values in the 'landingPad' column: {missing_va
lues}")

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__get
item__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.ge
t_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 'landing_pad'
```

In [81]:
```
import requests
import pandas as pd

# Step 1: Send GET request to the SpaceX API
url = "https://api.spacexdata.com/v4/launches"  # SpaceX API URL for launches
response = requests.get(url)

# Step 2: Convert the response JSON into a DataFrame
launch_data = response.json()
df = pd.json_normalize(launch_data)

# Step 3: Inspect the column names
print(df.columns)  # Print all column names to see if landingPad is present or l

# Step 4: Check for missing values in the landingPad column (adjust column name
# Uncomment and adjust this after you confirm the correct column name
# missing_values = df['landing_pad'].isna().sum()
# print(f"Number of missing values in the 'landingPad' column: {missing_values}"
```

```
Index(['static_fire_date_utc', 'static_fire_date_unix', 'net', 'window',
       'rocket', 'success', 'failures', 'details', 'crew', 'ships', 'capsules',
       'payloads', 'launchpad', 'flight_number', 'name', 'date_utc',
       'date_unix', 'date_local', 'date_precision', 'upcoming', 'cores',
       'auto_update', 'tbd', 'launch_library_id', 'id', 'fairings.reused',
       'fairings.recovery_attempt', 'fairings.recovered', 'fairings.ships',
       'links.patch.small', 'links.patch.large', 'links.reddit.campaign',
       'links.reddit.launch', 'links.reddit.media', 'links.reddit.recovery',
       'links.flickr.small', 'links.flickr.original', 'links.presskit',
       'links.webcast', 'links.youtube_id', 'links.article', 'links.wikipedia',
       'fairings'],
      dtype='object')
```

In [82]:
```
# Check for missing values in the 'launchpad' column
missing_values_launchpad = df['launchpad'].isna().sum()
print(f"Number of missing values in the 'launchpad' column: {missing_values_lau

# Inspect the 'launchpad' column for the first few rows to understand its conte
print(df['launchpad'].head())
```

```
Number of missing values in the 'launchpad' column: 0
0    5e9e4502f5090995de566f86
1    5e9e4502f5090995de566f86
2    5e9e4502f5090995de566f86
3    5e9e4502f5090995de566f86
4    5e9e4502f5090995de566f86
Name: launchpad, dtype: object
```

In [83]:
```
# Count the missing values in the 'landingPad' column
missing_values = df['landingPad'].isnull().sum()

# Print the number of missing values
print(f"Number of missing values in the 'landingPad' column: {missing_values}")
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.ge
t_loc(self, key)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\\_libs\\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

File pandas\\_libs\\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

KeyError: 'landingPad'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[83], line 2
      1 # Count the missing values in the 'landingPad' column
----> 2 missing_values = df['landingPad'].isnull().sum()
      4 # Print the number of missing values
      5 print(f"Number of missing values in the 'landingPad' column: {missing_va
lues}")

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__get
item__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.ge
t_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 'landingPad'
```

In [84]:
```
# Print the column names to check for any differences
print(df.columns)
```

```
Index(['static_fire_date_utc', 'static_fire_date_unix', 'net', 'window',
       'rocket', 'success', 'failures', 'details', 'crew', 'ships', 'capsules',
       'payloads', 'launchpad', 'flight_number', 'name', 'date_utc',
       'date_unix', 'date_local', 'date_precision', 'upcoming', 'cores',
       'auto_update', 'tbd', 'launch_library_id', 'id', 'fairings.reused',
       'fairings.recovery_attempt', 'fairings.recovered', 'fairings.ships',
       'links.patch.small', 'links.patch.large', 'links.reddit.campaign',
       'links.reddit.launch', 'links.reddit.media', 'links.reddit.recovery',
       'links.flickr.small', 'links.flickr.original', 'links.presskit',
       'links.webcast', 'links.youtube_id', 'links.article', 'links.wikipedia',
       'fairings'],
      dtype='object')
```

In [85]:
```
# Expand the nested 'cores' column
core_data = pd.json_normalize(df['cores'])

# Print the columns in the normalized core data
print(core_data.columns)
```

```
RangeIndex(start=0, stop=3, step=1)
```

In [86]:
```
# Step 1: Normalize the nested cores data for all rows
core_data = pd.json_normalize(df['cores'].explode())

# Step 2: Check available columns
print(core_data.columns)
```

```
Index(['core', 'flight', 'gridfins', 'legs', 'reused', 'landing_attempt',
       'landing_success', 'landing_type', 'landpad'],
      dtype='object')
```

In [87]:
```
# Count the missing values in the 'landpad' column
missing_values = core_data['landpad'].isnull().sum()

print(f"Number of missing values in the 'landpad' column: {missing_values}")
```

```
Number of missing values in the 'landpad' column: 58
```

In [88]:
```
# Count the number of launches per site
launch_counts = df['LaunchSite'].value_counts()

# Display the result
print(launch_counts)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.ge
t_loc(self, key)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\\_libs\\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

File pandas\\_libs\\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.P
yObjectHashTable.get_item()

KeyError: 'LaunchSite'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[88], line 2
      1 # Count the number of launches per site
----> 2 launch_counts = df['LaunchSite'].value_counts()
      4 # Display the result
      5 print(launch_counts)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__get
item__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.ge
t_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 'LaunchSite'
```

In [89]:
```
df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cl
df.head(10)
```

Out[89]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	Nor Nor
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	Nor Nor
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	Nor Nor
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	Fal Oce
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	Nor Nor
5	6	2014-01-06	Falcon 9	3325.000000	GTO	CCAFS SLC 40	Nor Nor
6	7	2014-04-18	Falcon 9	2296.000000	ISS	CCAFS SLC 40	Tru Oce
7	8	2014-07-14	Falcon 9	1316.000000	LEO	CCAFS SLC 40	Tru Oce
8	9	2014-08-05	Falcon 9	4535.000000	GTO	CCAFS SLC 40	Nor Nor
9	10	2014-09-07	Falcon 9	4428.000000	GTO	CCAFS SLC 40	Nor Nor

In [90]:
```
df.isnull().sum()/len(df)*100
```

Out[90]:
```
FlightNumber       0.000000
Date               0.000000
BoosterVersion     0.000000
PayloadMass        0.000000
Orbit              0.000000
LaunchSite         0.000000
Outcome            0.000000
Flights            0.000000
GridFins           0.000000
Reused             0.000000
Legs               0.000000
LandingPad        28.888889
Block              0.000000
ReusedCount        0.000000
Serial             0.000000
Longitude          0.000000
Latitude           0.000000
dtype: float64
```

In [91]:
```
df.dtypes
```

Out[91]:
```
FlightNumber        int64
Date               object
BoosterVersion     object
PayloadMass       float64
Orbit              object
LaunchSite         object
Outcome            object
Flights             int64
GridFins             bool
Reused               bool
Legs                 bool
LandingPad         object
Block             float64
ReusedCount         int64
Serial             object
Longitude         float64
Latitude          float64
dtype: object
```

In [92]:
```
# Apply value_counts() on column LaunchSite
# Count the number of launches per site
launch_counts = df['LaunchSite'].value_counts()

# Display the result
print(launch_counts)
```

```
LaunchSite
CCAFS SLC 40    55
KSC LC 39A      22
VAFB SLC 4E     13
Name: count, dtype: int64
```

In [93]:
```
# Count the number and occurrence of each orbit type
orbit_counts = df['Orbit'].value_counts()

# Display the result
print(orbit_counts)
```

```
Orbit
GTO      27
ISS      21
VLEO     14
PO        9
LEO       7
SSO       5
MEO       3
ES-L1     1
HEO       1
SO        1
GEO       1
Name: count, dtype: int64
```

In [94]:
```
# Count the number and occurrence of each landing outcome
landing_outcomes = df['Outcome'].value_counts()

# Display the result
print(landing_outcomes)
```

```
Outcome
True ASDS      41
None None      19
True RTLS      14
False ASDS      6
True Ocean      5
False Ocean     2
None ASDS       2
False RTLS      1
Name: count, dtype: int64
```

In [95]:
```
for i,outcome in enumerate(landing_outcomes.keys()):
    print(i,outcome)
```

```
0 True ASDS
1 None None
2 True RTLS
3 False ASDS
4 True Ocean
5 False Ocean
6 None ASDS
7 False RTLS
```

In [96]:
```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])
bad_outcomes
```

Out[96]:
```
{'False ASDS', 'False Ocean', 'False RTLS', 'None ASDS', 'None None'}
```

In [97]:
```
# Create landing_class based on whether the outcome is in bad_outcomes
landing_class = [0 if outcome in bad_outcomes else 1 for outcome in df['Outcome
```

In [98]:
```
df['Class']=landing_class
df[['Class']].head(8)
```

Out[98]:

	Class
0	0
1	0
2	0
3	0
4	0
5	0
6	1
7	1

In [99]:
```
success_rate = df['Class'].mean()
print(f"Landing success rate: {success_rate:.2%}")
```

```
Landing success rate: 66.67%
```

In [100…
```
df.to_csv("dataset_part_2.csv", index=False)
```

In [ ]: